

CampusFix AI: Technical Whitepaper

Languages, System Workflows, Backend Architecture & Engineering Problem Solutions

Core Languages: Python 3.13, JavaScript/TypeScript, SQL, HTML5/CSS3

Backend: FastAPI (Async ASGI) + Uvicorn + SQLite3 Engine

Frontend: React 18 + Vite 5 + Tailwind CSS + Lucide Icons

AI Engine: Gemini 1.5 Flash Vision + Dual-Mode Offline Fallback

1. Executive Technical Summary

CampusFix AI is an evidence-based visual intelligence system engineered to eliminate text-heavy, ambiguous campus complaint forms. The platform implements a complete digital lifecycle: converting raw smartphone photographs of damaged infrastructure into structured, deduplicated, and auto-routed maintenance work orders. Most importantly, it introduces the industry's first **AI-assisted Before/After visual resolution verification loop**, ensuring maintenance tickets cannot be closed without objective photographic proof of completed repairs.

The system is built on a **decoupled client-server architecture**: an asynchronous Python **FastAPI** backend orchestrates multi-agent vision routines and localized SQLite transactions, while a high-performance **React 18 + Vite** frontend delivers real-time canvas overlays, interactive room scanning, and supervisor verification dashboards.

2. Full-Stack Technology & Language Matrix

System Layer	Languages & Runtimes	Key Frameworks & Libraries	Architectural Purpose & Rationale
Backend Server	Python 3.13 (AsyncIO, Type Hints)	FastAPI , Uvicorn, Pydantic, Python-Multipart, CORS Middleware	High-concurrency async endpoint execution, automatic OpenAPI schema generation, and sub-15ms route overhead.
Frontend Client	JavaScript (ES2022), HTML5, CSS3	React 18 , Vite 5, Tailwind CSS 3.4, Lucide-React, HTML5 Canvas API	Single-Page Application (SPA) with hot-module reloading, responsive grid layout, and real-time bounding box canvas drawing.
AI & Computer Vision	RESTful JSON APIs, Base64 Multimodal	Google Gemini 1.5 Flash Vision , JSON-Schema Prompting, Urllib Client	Zero-shot multimodal defect detection, department classification, priority scoring, and before/after repair image comparison.
Database & Storage	SQL (ANSI standard)	SQLite3 (WAL mode), JSON serialization, Row Factory, Audit Ledger	Embedded, zero-maintenance relational persistence with strict foreign-key integrity and complete audit-log tracking.
Tooling & Deployment	Bash / Shell Scripting	Bash (start.sh, run.sh), Node.js / NPM, ReportLab PDF Engine	One-click automated launch sequence, dependency bootstrap, and automated executive documentation rendering.

3. End-to-End System Workflow (The 7-Stage Pipeline)

CampusFix AI replaces manual paper/text workflows with an automated, closed-loop evidence pipeline:

CAPTURE: Student snaps photo → **UNDERSTAND:** Vision model detects objects & defects → **CLASSIFY:** Auto-categorizes to Electrical/Plumbing/IT → **ROUTE:** Dispatches task to department queue → **REPAIR:** Technician fixes fault & snaps 'after' photo → **VERIFY:** AI validates physical repair match → **CLOSE:** Human supervisor signs off.

4. Backend Deep-Dive: How It Was Achieved

The backend is structured as a modular, service-oriented Python FastAPI architecture organized into distinct functional layers:

- **Modular Router Hierarchy:** All endpoints are strictly decoupled into 4 dedicated routers in `app/routers/`: `incidents.py` (ingestion, room-scan, upload), `maintenance.py` (technician queues, status lifecycle), `verification.py` (before/after image processing), and `analytics.py` (workload, SLA metrics).
- **Dual-Layer State Machine:** Every incident transitions through a rigid, validated lifecycle: Reported → Reviewed → Assigned → In Progress → Awaiting Verification → Verified Resolved. State transitions cannot skip steps without required evidence.
- **Relational Schema with Audit Logs:** The database engine (`database.py`) utilizes SQLite with custom connection pooling. Every ticket update triggers an automated audit trail in the `audit_logs` table recording the actor (Student, Technician, Supervisor), action, timestamp, and metadata.
- **Static Asset & Demo Pipeline:** A dedicated asset pipeline (`app.config.DEMO_ASSETS_DIR`) serves high-resolution sample before/after image pairs directly over HTTP, enabling instant offline testing and predictable stage presentations.

Database Schema Specification (campusfix.db)

Column Name	Data Type	Constraints & Indexing	Functional Role in System
id	TEXT	PRIMARY KEY (e.g. 'CF-001')	Unique deterministic incident identifier.
category / department	TEXT	NOT NULL, Indexed	Electrical, Plumbing, Carpentry, IT, Housekeeping.
before_image / after_image	TEXT (LONG)	Nullable, Base64/URI	Cryptographically immutable before & after photographic evidence.
duplicate_of	TEXT	FOREIGN KEY → incidents(id)	Links duplicate submissions to primary parent incident.
verification_status	TEXT	DEFAULT 'Unverified'	Stores AI result: 'CONSISTENT_VERIFIED' or 'ANOMALY_FLAGGED'.

5. AI & Computer Vision Engineering: The Three Agents

CampusFix AI implements three specialized AI agents located in `backend/app/ai/` to handle visual cognition, similarity deduplication, and resolution verification:

A. Multimodal Vision Agent (`vision_agent.py`)

- **Zero-Shot Defect Extraction:** Sends base64-encoded classroom images to the Gemini 1.5 Flash multimodal endpoint using strict JSON-schema decoding with low temperature ($T=0.2$).
- **Room Scan Batching Mode:** Identifies multiple simultaneous physical defects in a single wide-angle photograph (e.g. broken ceiling fan + damaged study desk + cracked window pane) with normalized bounding coordinates `[ymin, xmin, ymax, xmax]`.
- **Deterministic Department Classification:** Automatically maps detected problems to the appropriate operational unit (e.g., electrical sparks → Electrical Maintenance; leaking pipe → Plumbing).

B. Composite Duplicate Detection Agent (`duplicate_agent.py`)

- **Multi-Factor Similarity Formula:** Evaluates incoming incident telemetry against all active, unresolved campus tickets using a composite weighted similarity score:

Similarity Matching Formula:

$\text{Score} = 0.45 * (\text{Location_Overlap}) + 0.15 * (\text{Room_Number_Match}) + 0.20 * (\text{Category_Match}) + 0.30 * (\text{Object_Jaccard_Sim})$

If $\text{Score} \geq 0.65$ (65% confidence), the system flags a duplicate, preventing redundant ticket creation and incrementing subscriber upvotes.

C. Before/After Resolution Verification Agent (`verification_agent.py`)

- **Dual-Image Differential Analysis:** Ingests both the original 'Before' photographic evidence and the technician's 'After' repair photograph into a unified multimodal context window.
- **Environmental & Physical Consistency Check:** Confirms that the background surroundings match the original location while verifying that the physical defect (e.g. dangling fan blade, water pool) has been eliminated.
- **Supervisor Decision Support:** Generates a confidence rating (80%–99%) and human-readable engineering verification notes (e.g., *'New standard fan blades installed and aligned; background ceiling matches CF-001'*).

D. Dual-Mode Architecture & Offline Fallback Reliability

To guarantee **100% demo uptime on stage** regardless of venue Wi-Fi reliability or API quota limits, the system features an intelligent fallback layer. If `GEMINI_API_KEY` is unset or the external API times out, the backend seamlessly switches to a deterministic smart heuristic simulator, ensuring instant, glitch-free execution during live evaluations.

6. Engineering Challenges Faced & Solutions Implemented

During the design and construction of CampusFix AI, several critical real-world edge cases were identified and solved:

Technical Challenge Faced	Underlying Root Cause	Engineering Method Used to Solve It
1. Poor Image Quality & Odd Camera Angles	Student photos vary widely in lighting, angle, and camera blur.	Enforced confidence scoring thresholds. If detection confidence is below 70%, the ticket is flagged for human-in-the-loop review rather than guessing.
2. Duplicate Complaint Avalanches	A single broken pipe triggers 15 separate tickets, overwhelming technicians.	Engineered multi-vector similarity in <code>duplicate_agent.py</code> combining room-number regex parsing and Jaccard token overlap to auto-merge tickets.
3. Fake Technician Closures ('Ghost Repairs')	Contractors routinely mark tickets 'Resolved' without fixing the problem.	Created <code>verification_agent.py</code> : Ticket closure is physically blocked until an after-repair photo is uploaded and verified by AI comparison.
4. Multi-Defect Classroom Complexity	Single rooms often contain broken lights, fans, and desks simultaneously.	Implemented Room Scan Mode : Generates multiple candidate bounding boxes in one shot, allowing students to selectively submit individual issues.
5. Zero Capex / Zero Hardware Mandate	Colleges refuse to buy expensive dedicated IoT sensor infrastructure.	Built a 100% software-driven web pipeline utilizing existing student smartphone cameras and HTML5 Canvas API—requiring zero hardware investment.

7. RESTful API Endpoint Architecture

Endpoint Route	HTTP	Payload Parameters	Response & Action Executed
<code>/api/incidents/analyze-single</code>	POST	<code>image_data</code> (b64), <code>location</code>	Analyzes defect, classifies category, checks duplicates.
<code>/api/incidents/analyze-room-scan</code>	POST	<code>image_data</code> (b64), <code>location</code>	Surfaces multiple candidate defects with bounding coordinates.
<code>/api/maintenance/assign</code>	POST	<code>incident_id</code> , <code>staff_name</code>	Dispatches work order; updates status to 'Assigned'.
<code>/api/verification/verify-repair</code>	POST	<code>incident_id</code> , <code>after_image</code>	Executes Before/After AI comparison and logs audit trail.
<code>/api/analytics/dashboard</code>	GET	None (query params)	Aggregates SLA compliance, open tickets, and department workload.